INTRODUCTION TO INFRASTRUCTURE AS CODE (IAC) WITH TERRAFORM

TERRAFORM

1. It is a free and open-source Infrastructure as Code (IaC) tool used to provision and manage infrastructure safely and predictably on any cloud platform.
2. It is developed by HashiCorp.
3. It uses HCL (HashiCorp Configuration Language) for writing infrastructure code

---

HISTORY

1. It was first introduced by Mitchell Hashimoto in July 2014, initially supporting AWS and DigitalOcean.
2. Later, in 2017, Microsoft partnered with Terraform to help create and manage Azure resources using Terraform.
3. Over time, many cloud providers adopted Terraform, and today it supports multi-cloud and hybrid cloud environments.
4. Terraform uses HCL (HashiCorp Configuration Language), which is similar to JSON but easier and more human-readable.

CLOUD-SPECIFIC IAC TOOLS

- AWS → AWS CloudFormation
- GCP → Google Cloud Deployment Manager
- Azure → Azure Resource Manager

DIFFERENCE BETWEEN CONFIGURATION MANAGEMENT TOOL AND IAC TOOL

Configuration Management Tools

1. Used to install and manage software on existing servers.
2. Focus on server configuration and application setup.
3. Examples: Ansible, Puppet, Chef
4. Tasks include installing packages, managing services, and updating configuration files.

Infrastructure as Code (IaC) Tools

1. Used to provision and manage infrastructure resources.
2. Focus on creating infrastructure like servers, networks, and databases.
3. Examples: Terraform, AWS CloudFormation
4. Tasks include creating EC2 instances, VPCs, subnets, and load balancers.

BENEFITS OF TERRAFORM

1. It supports multiple cloud providers like AWS, Azure, and GCP.
2. It is open-source and free to use.
3. Infrastructure code can be reused to create the same setup multiple times.
4. It easily integrates with configuration management tools like Ansible and Chef.
5. It is highly extensible through plugins and providers.

DATA TYPES

1. **String** → Used to store text values.
   Example: "user1", "user2", "user3"

2. **Number** → Used to store numeric values.
   Example: 123, 456, 789

3. **Boolean** → Used for true or false conditions.
   Example: true, false

4. **List** → Used to store multiple values in order.
   Example: ["apple", "banana"]

5. **Map/Dictionary** → Used to store key-value pairs.
   Example:
   ```
   {
    city = "Pune"
    state = "MH"
   }
   ```

TERRAFROM BLOCKS

1. Provider
2. Resource
3. Variables
4. Output
5. Backend
6. Provisionar blocks
7. Module
8. Data

TWO APPROACHES TO WRITE A CODE

1. IMPERATIVE APPROACH

1. You write step-by-step instructions.
2. You tell the system HOW to do something.
3. Common in shell scripting or manual cloud commands.
4. Example:

```
aws ec2 run-instances \
--image-id ami-123456 \
--instance-type t2.micro
```

you are giving direct commands:

- Create EC2
- Use this AMI
- Use this instance type

2.DECLARATIVE APPROACH

1. You define the desired final state.
2. Terraform decides HOW to achieve it.
3. Terraform is mainly a Declarative IaC Tool.
4. Example:

```
resource "aws_instance" "web" {
 ami           = "ami-123456"
 instance_type = "t2.micro"
}
```

Terraform automatically:

- checks current state
- creates resources
- updates changes
- destroys if needed
